

JAVA OBJECT-ORIENTED PROGRAMMING LAB REPORT

Complete Implementation & Documentation: Classes, Encapsulation, Inheritance, Overloading, Abstraction & Interfaces

Course: Object-Oriented Programming with Java

Document Type: Comprehensive Lab Report

1. SYSTEM REQUIREMENTS

To compile and run these Java OOP modules, the following hardware and software specifications are required:

Component Category	Minimum / Recommended Specification
Operating System	Windows 10/11 (64-bit), macOS 11.0+, or Linux (Ubuntu 20.04 LTS)
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or higher
Memory (RAM)	Minimum 4 GB RAM (8 GB recommended)
Java Development Kit (JDK)	Java SE 8 / 11 / 17 / 21 LTS (`javac` compiler & `java` runtime)
Development Environment	Command Line Interface (CMD/Terminal) or IDE (VS Code, IntelliJ IDEA, Eclipse)

2. COMPILATION & EXECUTION PROCEDURE

Step-by-Step CLI Instructions:

1. **Save Source Files:** Save source files with matching public class names:

- Task2.java (Classes, Objects & Encapsulation)
- Task21.java (Hierarchical Inheritance & Method Overloading)
- Task23.java (Abstract Classes & Interfaces)

2. **Compile Source Files:**

```
javac Task2.java Task21.java Task23.java
```

3. **Execute Bytecode:**

```
Run Task 2: java Task2
```

```
Run Task 21: java Task21
```

```
Run Task 23: java Task23
```

3. MODULE 1: CLASSES, OBJECTS & ENCAPSULATION (TASK2)

Task 1.1: Basic Class and Object Creation

Objective: Instantiate objects from a custom class, assign field values directly, and display details using member methods.

Algorithm:

1. Define class book with attributes bookid, bookname, author, and price.
2. Define method display() to print all field values.
3. In main(), instantiate object b, assign attributes, and call b.display().
4. Instantiate object l, assign attributes, and call l.display().

UML Class Diagram

```
+-----+
|               book               |
+-----+
| + bookid: int                    |
| + bookname: String               |
| + author: String                 |
| + price: double                  |
+-----+
| + display(): void                |
+-----+
```

Source Code

```
class book {
    int bookid;
    String bookname;
    String author;
    double price;

    void display() {
        System.out.println("book id name" + bookid);
        System.out.println("bookname is" + bookname);
        System.out.println("book authoris " + author);
        System.out.println("the price of book is" + price);
    }
}

public class Task2 {
    public static void main(String[] args) {
        book b = new book();
        b.bookid = 244;
        b.bookname = "java learns";
        b.author = "james gosling";
        b.price = 250.00;

        book l = new book();
        l.bookid = 28;
        l.bookname = "learn";
        l.author = " suresh";
    }
}
```

```

        l.price = 45.50;

        b.display();
        l.display();
    }
}

```

Execution Output

```

book id name244
bookname isjava learns
book authoris james gosling
the price of book is250.0
book id name28
bookname islearn
book authoris suresh
the price of book is45.5

```

Task 1.2: Encapsulation with Private Fields & Getters/Setters

Objective: Secure data access using private member variables, parameterized constructors, and accessor/mutator methods.

Algorithm:

1. Declare private fields in class book.
2. Provide parameterized constructor to initialize fields using this keyword.
3. Implement getter and setter methods for each private attribute.
4. In main(), construct object c and access data safely via getters.

Source Code

```

class book {
    private int bookid;
    private String bookname;
    private String author;
    private double price;

    public book(int id, String name, String author, double price) {
        this.bookid = id;
        this.bookname = name;
        this.author = author;
        this.price = price;
    }

    void setbookid(int id) { bookid = id; }
    void setbookname(String name) { bookname = name; }
    void setauthor(String author) { this.author = author; }
    void setprice(double price) { this.price = price; }

    int getbookid() { return bookid; }
    String getbookname() { return bookname; }
    String getauthor() { return author; }
    double getprice() { return price; }
}

```

```
public class Task2 {  
    public static void main(String[] args) {  
        book c = new book(244, "java", "james", 350.50);  
        System.out.println("book id is " + c.getbookid());  
        System.out.println("book name is " + c.getbookname());  
        System.out.println("author is " + c.getauthor());  
        System.out.println("price is " + c.getprice());  
    }  
}
```

Execution Output

```
book id is 244  
book name is java  
author is james  
price is 350.5
```

4. MODULE 2: INHERITANCE & POLYMORPHISM (TASK21)

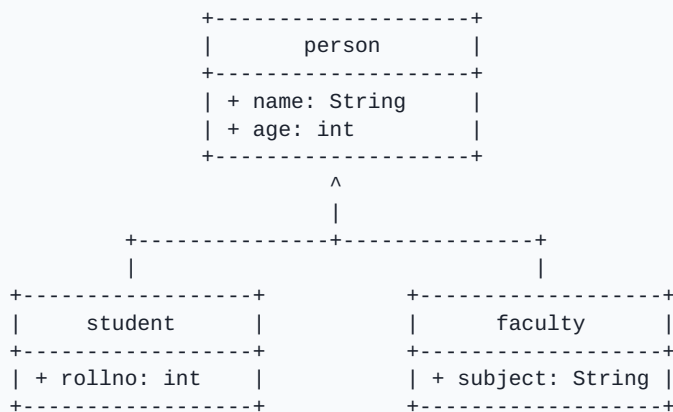
Task 2.1: Hierarchical Inheritance

Objective: Implement hierarchical inheritance where multiple child classes (student, faculty) inherit common attributes from a single base class (person).

Algorithm:

1. Define superclass person with name, age, and method getperson().
2. Derive subclass student with rollno and method show().
3. Derive subclass faculty with subject and method show().
4. In main(), instantiate student and faculty, set properties, and invoke show().

UML Inheritance Hierarchy Diagram



Source Code

```
class person {
    String name;
    int age;

    void getperson(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

class student extends person {
    int rollno;

    void getstudent(int rollno) {
        this.rollno = rollno;
    }

    void show() {
        System.out.println("---student details---");
        System.out.println("name is " + name);
        System.out.println("age is" + age);
        System.out.println("rollno is" + rollno);
    }
}
```

```

    }
}

class faculty extends person {
    String subject;

    void getfaculty(String subject) {
        this.subject = subject;
    }

    void show() {
        System.out.println("--- faculty details---");
        System.out.println("name of faculty is" + name);
        System.out.println(" age of " + age);
        System.out.println("subject is " + subject);
    }
}

public class Task21 {
    public static void main(String[] args) {
        student s = new student();
        s.getperson("hemanth", 19);
        s.getstudent(244);

        faculty f = new faculty();
        f.getperson("srinu", 35);
        f.getfaculty("programming");

        s.show();
        f.show();
    }
}

```

Execution Output

```

---student details---
name is hemanth
age is19
rollno is244
--- faculty details---
name of faculty is dynamic srinu
age of 35
subject is programming

```

Task 2.2: Compile-Time Polymorphism (Method Overloading)

Objective: Demonstrate method overloading by defining multiple `area()` methods with different parameter lists within an `Area` class.

Algorithm: Overload `area()` method: (1) `area(double radius)` for circle, (2) `area(int length, int breadth)` for rectangle, (3) `area(double b, double h)` for triangle.

Source Code & Output

```

public class Task21 {
    static class Area {
        double area(double radius) {

```

```

        return 3.14 * radius * radius;
    }

    int area(int length, int breadth) {
        return length * breadth;
    }

    double area(double b, double h) {
        return 0.5 * b * h;
    }
}

public static void main(String[] args) {
    Area a = new Area();
    System.out.println("area of circle " + a.area(5.0));
    System.out.println("area of rectangle " + a.area(10, 5));
    System.out.println("area of triangle " + a.area(3.0, 5.0));
}
}

```

```

area of circle 78.5
area of rectangle 50
area of triangle 7.5

```

5. MODULE 3: ABSTRACTION & INTERFACES (TASK23)

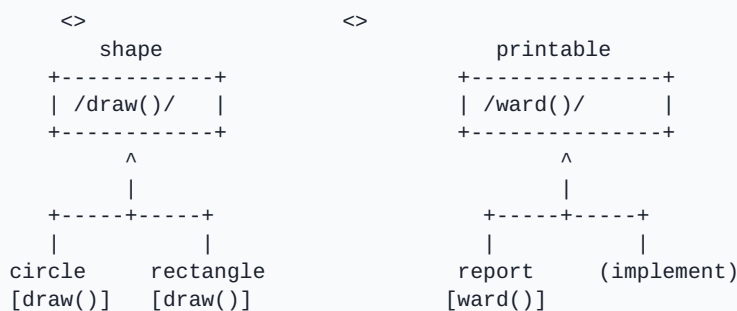
Task 3.1: Abstract Classes & Interfaces Implementation

Objective: Combine abstract classes and interfaces to achieve full abstraction and contract-based design in Java.

Algorithm:

1. Define abstract class shape with abstract method draw().
2. Implement concrete classes circle and rectangle overriding draw().
3. Define interface printable with abstract method ward().
4. Create class report implementing printable interface.
5. In main(), instantiate concrete classes and execute overridden behavior.

UML Abstract Class & Interface Structure



Source Code

```
abstract class shape {
    abstract void draw();
}

class circle extends shape {
    @Override
    public void draw() {
        System.out.println("the drawing is circle");
    }
}

class rectangle extends shape {
    @Override
    public void draw() {
        System.out.println("th edrawing is rectangle");
    }
}

interface printable {
    void ward();
}

class report implements printable {
    @Override
    public void ward() {
```

```
        System.out.println("the interface is impenents");
    }
}

public class Task23 {
    public static void main(String[] args) {
        report r = new report();
        r.ward();

        circle a = new circle();
        a.draw();

        rectangle t = new rectangle();
        t.draw();
    }
}
```

Execution Output

```
the interface is impenents
the drawing is circle
th edrawing is rectangle
```